



A Structured Benchmarking Dataset for TLA+ Specification Reasoning

Arslan Bisharat, Eric Spencer, , Brian Ortiz,
Khushboo Bhadauria, Anisa Ramos, TaiNing Wang, George K. Thiruvathukal,
Konstantin Läufer, Mohammed Abuhamad

INTRODUCTION

- **TLA+** is one of the most widely adopted specification languages in industry (AWS Dyanmo DB and S3, Azure Cosmos DB, etc.).
- Despite this industrial adoption, TLA+ specifications remain difficult and time-intensive to produce from natural language. The fundamental challenge is data scarcity: the entire public TLA+ corpus consists of only a few hundred modules
- Our survey of 144 research papers in this space reveals a pronounced asymmetry: Lean 4 has accumulated 68 unique datasets, Isabelle and Coq each have 9-11 datasets.
- TLA+, by contrast, has only 3 dedicated dataset, 2 dedicated models (both by a single author), and zero publicly released systems.

	Lean 4	Isabella	Coq	TLA+
Papers	32	5	5	4
Datasets	68	9	11	3
Models	210	21	26	2
Publicly released	21	3	2	0

Table 1: Neural theorem prover ecosystem coverage from 144 surveyed papers. TLA+ counts include only TLA+-specific studies; 10 additional general-purpose datasets appear in multi-prover studies including TLA+.

We introduce **TLA+Bench**, the first large-scale, open benchmark for TLA+ specification logic:

- Built on programmatically verifiable preferences using TLA+ parser (**SANY**) and model checker (**TLC**), serving as objective arbiters of correctness without the cost and subjectivity of human labels.

Contributions:

1. A benchmark dataset of 206 TLA+ specifications, each with a:
 1. A structured 73 field feature extraction
 2. An abstract syntax tree
 3. A paired natural language description
2. A proposed methodology to investigate “Logical Hivemind” hypothesis
3. A demonstratable pipeline for benchmark’s utility via Spec-Prover
4. A publicly released benchmarking dataset, extraction code, and evaluation scripts

METHODS

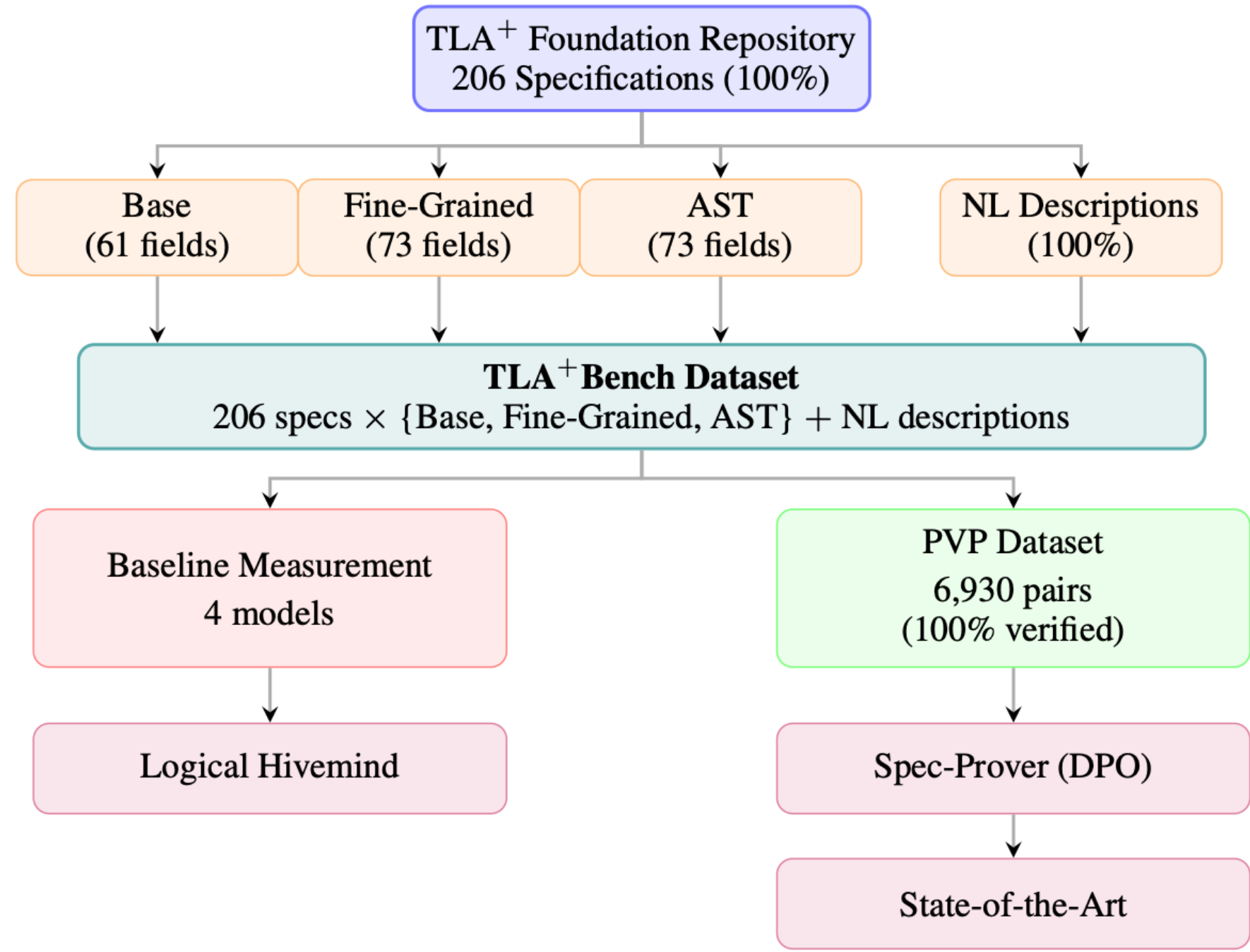


Figure 4: TLA+Bench system overview displays complete pipeline from 206 source specifications (100%) through three-level extraction to baseline measurement and Spec-Prover instruction with 6,930 programmatically verified preference pairs (100% correctness verification).

The corpus consists of 206 TLA+ specifications with a total of 73 extracted features per specification at the fine-grained level. The dataset produces 6,930 preference pairs after candidate measurement, that represent a substantial resource for preference-based model alignment.

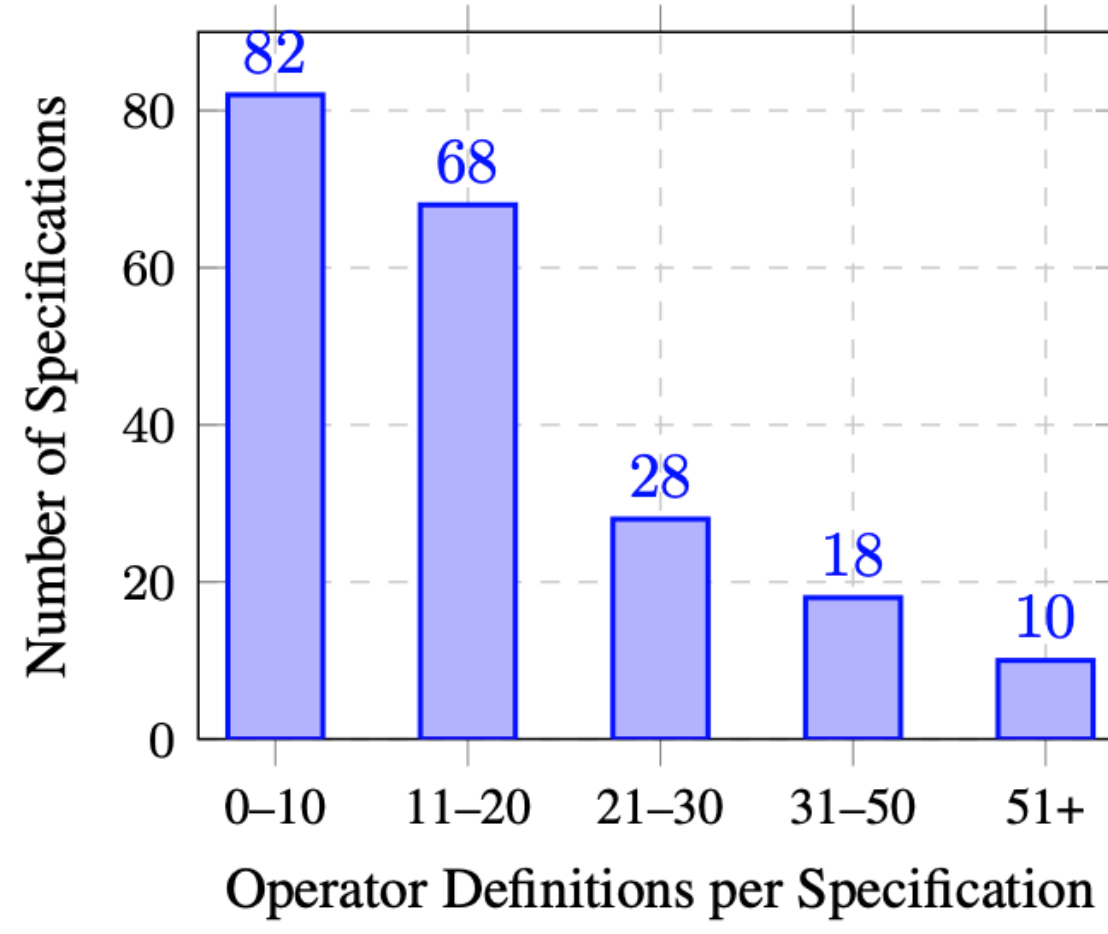


Figure 1: Distribution of structural complexity (operator definitions) across the 206 specifications. The long tail demonstrates substantial headroom.

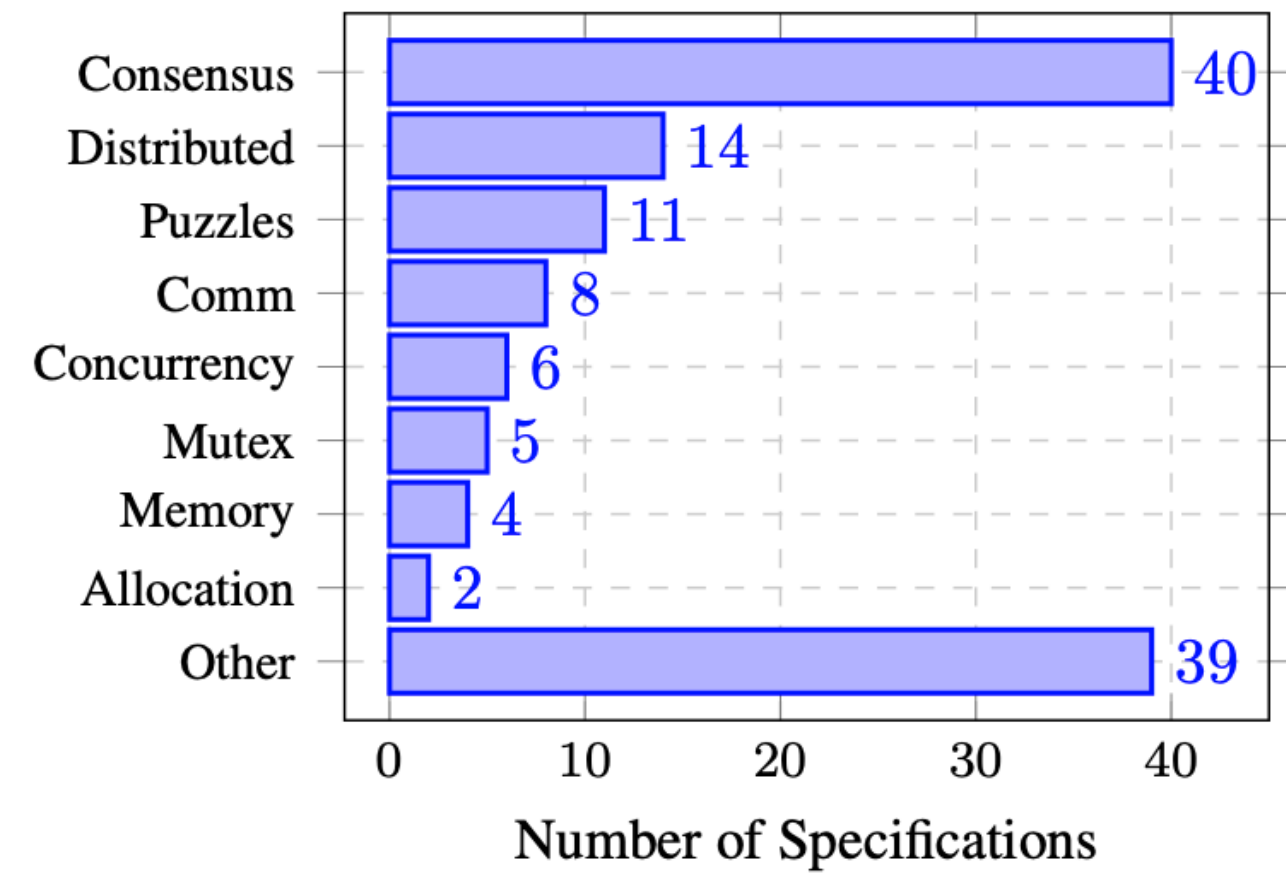
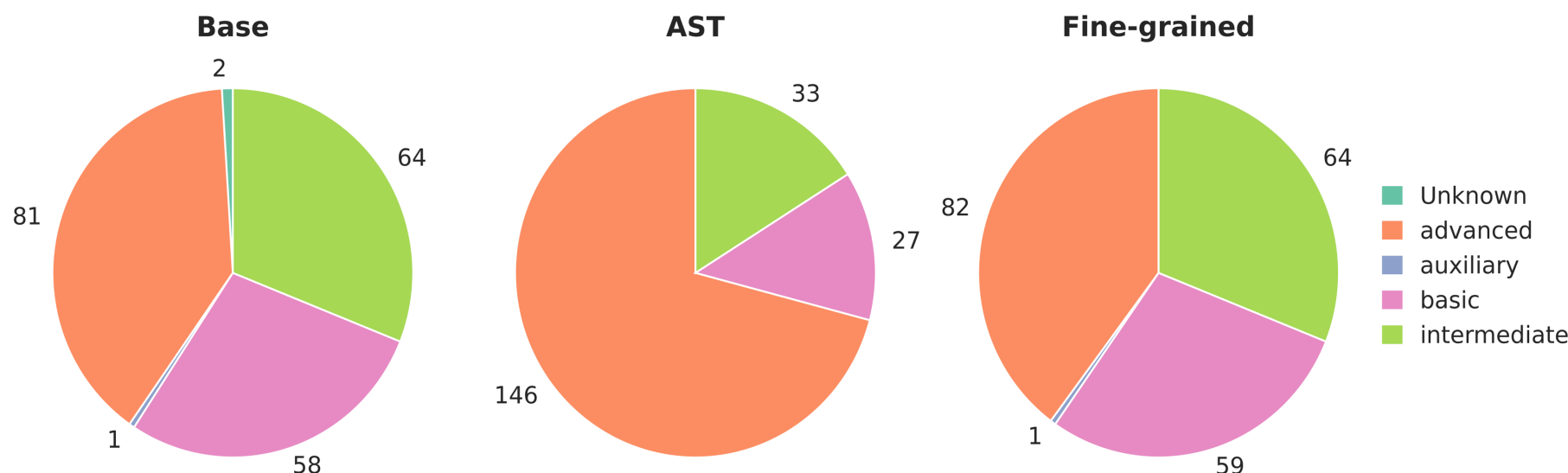


Figure 2: Domain coverage of TLA+Bench. Consensus and distributed algorithms represent the dominant industrial use cases.

RESULTS

- Total Specification: 206
 - Preference pairs (DPO instructions): 6,930
 - Instruction/validation/test split: 165/21/20
 - Mean description length (tokens): 517
 - Median description length (tokens): 516
- Specifications are assigned to four complexity tiers. Advanced specifications constitute 39.3% of the corpus (81 specs), intermediate represent 31.1% (64 specs) and basic account for 28.2% (58 specs), with a single auxiliary specification at 0.5%. This balanced distribution ensures evaluation across difficulty levels while maintaining a sufficient sample size per category.



Fine-Grained Extraction Example

Module-Level Constructs:

```
ModuleNames: Bakery
ExtendsModules: Naturals, Sequences
ConstantNames: [N]
VariableNames: [ticket, choice of, pc]
```

Operator Definitions (Subdivided):

```
OperatorDefNames_StatePredicate: [TypeInv]
OperatorDefNames_TemporalProperty: [Spec, fairness, AC1, AC2, AC3_1, AC3_2, AC4, AC5]
OperatorDefNames_Init: [Init]
OperatorDefNames_ValueDef: [parProg, coordProgA]
```

Logical and Relational Operators:

```
ConjunctionExpressions: 42
DisjunctionExpressions: 18
QuantifiedExpressions: 12
NegationExpressions: 8
```

Data Structure Constructs (Subdivided):

```
FunctionConstructor_ByDomain: [in participants l-> [...]]
FunctionConstructor_SetType: [[participants -> BOOLEAN],
[participants -> (wait, yes, no)], ...]
TupleExpressions: 15
SetConstruction: 8
```

Temporal and State Constructs:

```
NextStateRelation: 1
TemporalOperators: 5
FairnessConditions: 2
UnchangedExprs_Explicit: [ticket, choice of]
UnchangedExprs_VarRef: [pc]
```

Structural Metadata (New to Fine-Grained):

```
TemporalNesting: TRUE
VerificationTools: [TLC, TLAPS]
DivisionNotes: Reference implementation for mutual exclusion
```

Proof Constructs:

```
ProofCommands: 0
THEOREMStatements: 3
```

Model-Checker Constructs:

```
SymmetrySet: {1..N}
Invariants: [TypeInv]
Properties: [Spec]
```

AST Extraction Example

Module-Level Constructs (Auto-Extracted from SANY):

```
ModuleNames: Bakery
ExtendsModules: [Naturals, TLAPS]
ConstantNames: [N]
VariableNames: [num, flag, pc, unchecked, max, nxt]
```

Operator Definitions (Auto-Subdivided):

```
OperatorDefNames_StatePredicate: [Procs, vars, ProcSet, Next, MutualExclusion, ...]
OperatorDefNames_TemporalProperty: [Spec, DeadlockFree, StarvationFree, lSpec]
OperatorDefNames_Init: [Init]
OperatorDefNames_ValueDef: [p, Before, Attempts To, InCS]
```

Data Structure Constructs (SANY Node Classification):

```
FunctionConstructor_ByDomain: [[i in Procs l-> 0], [i in Procs l-> FALSE], [self in Procs l-> {}], ...]
FunctionConstructor_SetType: [[Procs -> Nat], [Procs -> BOOLEAN], [Procs -> SUBSET Procs], ...]
TupleConstructor: [< num, flag, pc, unchecked, max, nxt >, < num, flag, unchecked, max, nxt >, < unchecked, max, ... >]
```

Temporal and State Constructs (Auto-Classified):

```
UnchangedExprs_Explicit: [UNCHANGED < num, flag, unchecked, max, nxt >, UNCHANGED <unchecked, max>, UNCHANGED < num, nxt >, ...]
UnchangedExprs_VarRef: [UNCHANGED unchecked, UNCHANGED vars]
```

Proof Constructs:

```
ProofHints: [DEF Init, Inv, <21 DEF ncs, Inv, <22 DEF e1, ...]
ProofStructureSteps: [<1>, <1>1, <1>2, <1>3, <1>4, ...]
```

Structural Metadata (Auto-Detected):

```
TemporalNesting: False
VerificationTools: [TLAPS]
DivisionNotes: [Partition count: StatePredicate(8) + TemporalProperty(4) + Init(1) + ValueDef(4) + ActionDefNames(10) = 27 = OperatorDefNames total. Verified., ...]
```

FUTURE WORK

- Continue to build and expand TLA+Bench with more dataset samples
- Fine-tuning a larger model with TLA+Bench

REFERENCES

Arslan Bishirat, Brian Ortiz, Eric Spencer, Khushboo Bhadauria, Mohammed Abuhamad, Konstantin Läufer, George K. Thiruvathukal, TaiNing Wang Can LLMs Write Correct TLA+ Specifications? Evaluating Natural-Language-to-TLA+ Generation, Accepted to ICSOFT 2026, 2026.

Leslie Lamport, Markus A. Kuppe, Stephan Merz, et al. TLA+ Examples. URL <https://github.com/tlaplus/Examples>.

CONCLUSION

TLA+Bench, a large-scale, semantically rich dataset designed primarily as a rigorous evaluation benchmark for formal specification and verification of concurrent algorithms. By translating the entire TLA+ Examples repository into three tiers of structured schema, we provide the research community with a verifiable standard to measure the mathematical and temporal logic capabilities of language models. We propose a robust methodology to investigate the Logical Hivemind phenomenon, exploring whether top-tier language models systematically fail on identical bounds and complex temporal semantics despite architectural diversity. Beyond measurement, we outline how the benchmark can be utilized for model alignment. To circumvent the bottleneck of human verification data, we introduce Programmatically Verifiable Preferences (PVP). By utilizing the deterministic TLA+ toolchain as an automated preference oracle, we structured a pipeline to generate verified preference pairs without human labelers. We detail the methodology to employ this PVP data for Direct Preference Optimization to train Spec-Prover. TLA+Bench and the accompanying evaluation methodologies establish a rigorous foundation for formal methods research and pave the way to transition language models from simple generators into robust, mathematically grounded verification assistants.